Study book

# Topic 8: SQL basics

ProjectHub learning track: React/TypeScript to C#/.NET backend. Week 2, topic 8 of 20.

**Brief overview**

**What this lesson is about.** The relational database under ProjectHub, **by hand, without EF Core**: tables, rows and columns, **primary and foreign keys**, **constraints** (NOT NULL, UNIQUE, CHECK, DEFAULT), an index, and the ten basic queries (SELECT, WHERE, ORDER BY, paging, LIKE, NULL, INSERT, UPDATE, DELETE). Then we **break it on purpose** and read the errors.
**What we built.** `sql/01–05`: create the database, create the two tables, seed them with the same data as `SampleData.cs`, the ten queries, six statements the database refuses. Plus `scripts/run-sql.sh` (runs scripts with sqlcmd inside the container).
**Checkpoint:** query and change data without EF Core. Why it matters: in Topic 10 EF Core writes SQL **for** you; if you can't read SQL, EF is a black box.
**Your first time with Docker and DBeaver.** The setup had **8 problems** (Part 7). They are the most useful part of this book: almost all of them come down to one question, **which database is my query running in?**

## Contents


| Part | What is inside |
|---|---|
| 1 | A relational database in frontend terms; C# to SQL types |
| 2 | Keys: primary key, IDENTITY, foreign key, one-to-many; FK vs index (your question) |
| 3 | Constraints, collation, and why the database checks what C# already checks |
| 4 | Reading a query: execution order, NULL, ORDER BY, UPDATE/DELETE safety, SQL injection |
| 5 | Our scripts, line by line (01–05, run-sql.sh) |
| 6 | Real results: the ten queries and the six refused statements |
| 7 | Setup: Docker + DBeaver, and the 8 problems we hit (with causes and fixes) |
| 8 | Investigating without a debugger (the DEBUG tasks, with answers) |
| 9 | Your answers, corrections, interview Q&A |
| 10 | Summary, cheat sheet, checklist, what comes next |

# Part 1. A relational database in frontend terms

| Database | Closest JS/TS idea | ProjectHub |
|---|---|---|
| **database** | the whole app's data | `ProjectHubDb` |
| **table** | an array of objects that all have **exactly** the same shape | `Projects`, `Tasks` |
| **row** | one object in that array | the project "Website redesign" |
| **column** | a property, with a **fixed type** | `Name`, `CreatedAt` |
| **schema** (`dbo`) | a namespace / folder for tables | `dbo.Projects` ("database owner", the default) |

The big difference from a JS array: the database **enforces the shape and the rules** (types, required values, uniqueness, relationships) for **everyone** who writes to it, not just for your code.

## 1.1 Our C# entities map to tables

| C# (`Entities/`) | SQL |
|---|---|
| class `Project` | table `dbo.Projects` |
| one `Project` object | one row |
| property `Name` (`string`, required) | column `Name NVARCHAR(100) NOT NULL` |
| property `Description` (`string?`) | column `Description NVARCHAR(1000) NULL` |
| `List<TaskItem> Tasks` | **no column!** Each row in `Tasks` stores a `ProjectId` instead (Part 2) |

## 1.2 Column types: C# to SQL Server

| C# | SQL Server | Note |
|---|---|---|
| `int` / `long` | `INT` / `BIGINT` | |
| `string` | `NVARCHAR(n)` | **N** = Unicode (any language, emoji), like a C# string. `n` = max length in characters |
| `string` (long text) | `NVARCHAR(MAX)` | prefer a real limit |
| `DateTime` | `DATETIME2` | more precise than the old `DATETIME` |
| `bool` | `BIT` | 0 / 1 |
| `decimal` | `DECIMAL(18, 2)` | exact numbers: money (Topic 1) |
| `Guid` | `UNIQUEIDENTIFIER` | |
| `enum` | `INT` | `TaskItemStatus.Done` is stored as `2` (EF Core's default too) |
| `string?` / `int?` | same type + `NULL` | the column allows "no value" |

**NVARCHAR vs VARCHAR**

**NVARCHAR vs VARCHAR** (CHECK question 5, which you asked me to answer): `NVARCHAR` stores **Unicode** (2 bytes per character for most text: Cyrillic, Chinese, emoji, everything). `VARCHAR` stores 1 byte per character from a **code page** (one alphabet), so other languages turn into `?`. For user-entered text use `NVARCHAR` and write literals with the **N prefix**: `N'Mobile app'`. EF Core maps `string` to `NVARCHAR` by default.

# Part 2. Keys: how rows are identified and connected

## 2.1 Primary key and IDENTITY

**Primary key (PK):** the column that identifies each row: **unique and never NULL**. It's the entity identity from Topic 1 ("task 5 stays task 5 even after a rename").

```
Id INT IDENTITY(1, 1) NOT NULL,
CONSTRAINT PK_Projects PRIMARY KEY (Id)
```

`IDENTITY(1, 1)` = "the **database** generates the id: start at 1, add 1". It replaces our C# `_nextProjectId++` and the `lock` around it (Topic 4). Three consequences you must be able to say in an interview:

1. **It's atomic in the database**, so two servers inserting at the same moment never get the same id. Our `lock` only worked inside one process.
2. **You learn the id only after the INSERT**: `OUTPUT INSERTED.Id` in SQL (Q8), or EF Core fills `project.Id` for you after `SaveChanges` (Topic 10).
3. **Gaps are normal.** Failed inserts use up numbers, and deleted ids are never reused (C6 got **Id 7**, not 4). Never build logic on "ids are 1, 2, 3, ...".

## 2.2 Foreign key: the relationship is a number

```
ProjectId INT NOT NULL,
CONSTRAINT FK_Tasks_Projects_ProjectId FOREIGN KEY (ProjectId) REFERENCES dbo.Projects (Id)
```

- **One-to-many:** one project has many tasks. The FK lives on the **"many" side** (`Tasks.ProjectId`). `Projects` has **no** list of tasks: a column holds **one** value, never an array.
- "The tasks of project 1" = "rows in `Tasks` where `ProjectId = 1`". In C#, `project.Tasks` is a convenience. In the database, the relationship is just this number.
- The database **guarantees** every `ProjectId` points to an existing project (C2), and **refuses** to delete a project that still has tasks (C5). Alternative: `ON DELETE CASCADE` deletes the tasks too (EF Core's default for required relationships, Topic 10).

The FK points from the many side (Tasks) to the one side (Projects). Project 3 has no tasks, and that needs nothing stored.

```
Projects                        Tasks
+----+------------------+       +----+-----------+--------------------+--------+
| Id | Name             |       | Id | ProjectId | Title              | Status |
+----+------------------+       +----+-----------+--------------------+--------+
| 1  | Website redesign |<------+ 1  | 1         | Design mockups     | 2      |
| 2  | Mobile app       |<--+   | 2  | 1         | Build landing page | 1      |
| 3  | Internal tools   |   +---+ 5  | 2         | Login screen       | 1      |
+----+------------------+       +----+-----------+--------------------+--------+
```

## 2.3 Your question: "an FK is actually an index?"

You asked: *"FK (foreign key) is actually an index: the 'id' of a row of another table, so they are linked, right?"*

**The idea is right, the word is wrong.** An FK column does store the PK value of a row in another table (task 5 has `ProjectId = 2`, so it belongs to project 2; in JS: `{ id: 5, projectId: 2 }`). But an FK is **not an index**. On `Tasks.ProjectId` there are **three separate things**:

| Thing | What it is | Job |
|---|---|---|
| the **column** `ProjectId` | stores a number | data |
| the **foreign key** `FK_Tasks_Projects_ProjectId` | a **rule**: the value must exist in `Projects.Id`; blocks deleting a referenced project | **correctness** |
| the **index** `IX_Tasks_ProjectId` | a sorted lookup structure, like a book's index | **speed** (`WHERE ProjectId = 1` without scanning every row) |

**Remember**

SQL Server creates an index automatically for **PRIMARY KEY** and **UNIQUE**, but **not for a FOREIGN KEY**. That's why 02 creates `IX_Tasks_ProjectId` itself.
One-liner: **"A foreign key is a column holding another table's primary key value, plus a rule that the referenced row must exist."**

# Part 3. Constraints: rules the database enforces

| Constraint | Our example | What it stops |
|---|---|---|
| `NOT NULL` | `Name NVARCHAR(100) NOT NULL` | missing required values (C4, error 515) |
| `PRIMARY KEY` | `PK_Projects` | duplicate or empty ids |
| `FOREIGN KEY` | `FK_Tasks_Projects_ProjectId` | tasks pointing to nothing (C2); deleting a project that has tasks (C5); both error 547 |
| `UNIQUE` | `UQ_Projects_Name` | two projects with the same name, our Topic 3 rule (C1, error 2627) |
| `CHECK` | `CK_Tasks_Status CHECK (Status IN (0, 1, 2))` | invalid status numbers (C3, error 547) |
| `DEFAULT` | `DF_Projects_CreatedAt DEFAULT SYSUTCDATETIME()` | nothing; it **fills** a value when none is given (like `= DateTime.UtcNow`) |

The prefixes `PK_`, `FK_`, `UQ_`, `CK_`, `DF_`, `IX_` are a naming convention. When an error says `UQ_Projects_Name`, you instantly know **what kind** of rule broke and **where**. Unnamed constraints get random names like `UQ__Projects__737584F6...`.

## 3.1 Case-insensitive uniqueness for free

SQL Server's default **collation** (the rules for comparing text) is case-**in**sensitive, so `UNIQUE (Name)` treats `Mobile app` and `MOBILE APP` as the same. That's exactly our C# `StringComparison.OrdinalIgnoreCase` rule. C1 proves it: *"The duplicate key value is (MOBILE APP)"*. The same collation makes `LIKE N'%app%'` (Q6) case-insensitive.

## 3.2 "C# already validates. Why rules in the database too?"

1. **Other paths in.** Imports, other apps, a colleague's manual SQL. Topic 5's task #158 came from "the old tracker" and skipped our service's validation.
2. **Concurrency across servers.** Topic 4's `lock` protects inside **one process**. With 3 servers behind a load balancer, two requests can both pass the C# "is the name free?" check at the same moment. The UNIQUE constraint rejects the second INSERT (2627), whichever server it came from.
3. **The last line of defense.** Data lives for years; code changes every week.

**Your answer was half**

In CHECK you gave only reason 1 (*"no one can insert it even directly in the DB"*). An interviewer asking "why both?" wants **reason 2** too: the C# check and the INSERT are two separate steps, and between them another server can insert the same name. Only the database sees all inserts.

## 3.3 Index

`IX_Tasks_ProjectId`: a sorted lookup structure that makes `WHERE ProjectId = 1` fast. Trade-off: every INSERT/UPDATE also has to update the index, so you index columns you **search by often** ("the tasks of a project"), not everything. Topic 18 goes deeper (execution plans).

# Part 4. Reading a query

```
SELECT Id, Title, Status        -- 3. which columns to return (like .Select)
FROM dbo.Tasks                  -- 1. which table
WHERE ProjectId = 1             -- 2. which rows (like .Where)
  AND Status <> 2
ORDER BY CreatedAt DESC;        -- 4. in which order (like .OrderByDescending)
```

- **Written order is not execution order.** Logically the database does **FROM, WHERE, SELECT, ORDER BY**. LINQ's method chain (`tasks.Where(...).Select(...)`) is actually closer to how SQL **runs**.
- **Declarative:** you say **what** you want, not **how** (no loops). The database picks the strategy (use an index or scan the table).
- Syntax vs C#: `=` (not `==`), `<>` (not `!=`), `AND` / `OR` / `NOT`, strings in **single quotes** with `N`: `N'Mobile app'`. Comments start with `--`.
- `GO` isn't SQL. It's an instruction to the **tool** (sqlcmd, DBeaver, SSMS): "send everything above as one batch". `CREATE DATABASE` and a following `USE` must be in separate batches.

## 4.1 LINQ to SQL, side by side

| LINQ (Topic 2) | SQL |
|---|---|
| `.Where(t => t.ProjectId == 1)` | `WHERE ProjectId = 1` |
| `.Select(p => new { p.Id, p.Name })` | `SELECT Id, Name` |
| `.OrderByDescending(t => t.CreatedAt).ThenBy(t => t.Title)` | `ORDER BY CreatedAt DESC, Title ASC` |
| `.Skip(3).Take(3)` | `ORDER BY Id OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY` |
| `.Where(p => p.Name.Contains("app"))` | `WHERE Name LIKE N'%app%'` |
| `.Where(p => p.Description == null)` | `WHERE Description IS NULL` |
| `.FirstOrDefault(p => p.Id == 1)` | `SELECT TOP 1 ... WHERE Id = 1` |

## 4.2 NULL: the most common SQL trap

`NULL` means **"unknown / no value"**. It's not `''` and not `0`. SQL uses **three-valued logic**: a comparison is TRUE, FALSE or **UNKNOWN**. Any comparison with NULL is UNKNOWN (even `NULL = NULL`), and `WHERE` keeps only rows where the condition is **TRUE**. So `WHERE Description = NULL` matches **nothing**. Use `IS NULL` / `IS NOT NULL`.

The same trap hides in `<>`: `WHERE Description <> N'x'` sounds like "everything except x", but it **skips Mobile app** (its description is NULL, so `NULL <> 'x'` is UNKNOWN). Real run (DEBUG task 2): only projects 1 and 3 came back.

> **C# vs SQL**
> C# is different: in C#, `null == null` is `true`. When EF Core translates `p.Description == null` it writes `IS NULL` for you. When it translates `p.Description != "x"` it adds `OR Description IS NULL` to keep C# semantics. You'll see that in the generated SQL in Topic 11.

## 4.3 No ORDER BY = no order

A table is a **set**, not an array. Without `ORDER BY` the database returns rows in **whatever order is cheapest** for it. That order is **undefined**, not "random": it often looks stable in development, then changes when data grows, an index is added or the server is upgraded. In our run the last `SELECT` of 04 came back **3, 2, 1** (it read the name index). A UI shows a list? **Always** `ORDER BY`**.** Paging (`OFFSET/FETCH`) requires it.

## 4.4 UPDATE and DELETE: the WHERE is everything

```
UPDATE dbo.Projects SET Name = N'x';           -- no WHERE: EVERY project is renamed
DELETE FROM dbo.Tasks;                          -- no WHERE: EVERY task is gone
```

There's no undo. **The habit: run a** `SELECT` **with the same** `WHERE` **first**, check the rows and their count, *then* turn it into the `UPDATE`/`DELETE` (that's what Q9 does). Tools show "(N rows affected)": if N is not what you expected, stop. Topic 9 adds **transactions** (`BEGIN TRAN` ... `ROLLBACK`), a real safety net.

## 4.5 Security preview: SQL injection

```
var sql = "SELECT * FROM Projects WHERE Name = '" + userInput + "'";   // SQL injection
// userInput = x'; DROP TABLE Projects; --
// the database receives:  SELECT * FROM Projects WHERE Name = 'x'; DROP TABLE Projects; --'
```

The fix is **parameters**: the SQL text (`WHERE Name = @name`) and the values travel **separately**, so a value is never parsed as SQL. Escaping quotes by hand is **not** a fix. EF Core always sends parameters (you'll see `@p0` / `@__name_0` in the generated SQL, Topic 10).

# Part 5. Our scripts, line by line

In Rider they're in the `sql` solution folder. All five were run against SQL Server 2025 in the cloud container (Part 6 has the real output). Setup scripts 01–03 run through `scripts/run-sql.sh` (sqlcmd); 04 and 05 run in DBeaver, one statement at a time.

## 5.1 01-create-database.sql

```
-- Topic 8, script 1: create the database.
-- Run it with "Execute SQL Script" (the whole file), not "Execute SQL Statement".
-- Safe to run again: it only creates the database if it doesn't exist yet.

IF DB_ID(N'ProjectHubDb') IS NULL CREATE DATABASE ProjectHubDb;
GO

-- Check: the new database should be in this list (one row: ProjectHubDb).
SELECT name, create_date FROM sys.databases WHERE name = N'ProjectHubDb';
```

- `DB_ID(N'ProjectHubDb') IS NULL`: "does the database not exist yet?", so the script is **re-runnable** (idempotent).
- `GO` ends the batch: the following `SELECT` reads `sys.databases`, the server's catalog of databases.

## 5.2 02-create-tables.sql

```
-- Topic 8, script 2: create the two tables, by hand.
-- Run it with "Execute SQL Script" (the whole file).
-- Safe to run again: it DROPS both tables first, so all their data is deleted. Then run 03 to fill them again.
-- (No blank lines INSIDE a statement: some tools, like DBeaver, can treat a blank line as "end of statement".)

USE ProjectHubDb;
GO

-- Drop in reverse dependency order: Tasks references Projects, so Tasks must go first.
DROP TABLE IF EXISTS dbo.Tasks;
DROP TABLE IF EXISTS dbo.Projects;
GO

-- C#: Entities/Project.cs
CREATE TABLE dbo.Projects
(
    Id          INT            IDENTITY(1, 1) NOT NULL,   -- the database generates 1, 2, 3, ...
    Name        NVARCHAR(100)  NOT NULL,                  -- required string, max 100 (ProjectService.MaxNameLength)
    Description NVARCHAR(1000) NULL,                      -- string? -> NULL allowed
    CreatedAt   DATETIME2(7)   NOT NULL
        CONSTRAINT DF_Projects_CreatedAt DEFAULT SYSUTCDATETIME(),   -- like "= DateTime.UtcNow"
    CONSTRAINT PK_Projects PRIMARY KEY (Id),
    -- The business rule from Topic 3, now enforced by the database itself. The column uses the server's default
    -- case-INsensitive collation, so 'Mobile app' and 'MOBILE APP' count as the same name.
    CONSTRAINT UQ_Projects_Name UNIQUE (Name)
);
GO

-- C#: Entities/TaskItem.cs
CREATE TABLE dbo.Tasks
(
    Id        INT           IDENTITY(1, 1) NOT NULL,
    ProjectId INT           NOT NULL,                   -- which project this task belongs to
    Title     NVARCHAR(200) NOT NULL,
    Status    INT           NOT NULL
        CONSTRAINT DF_Tasks_Status DEFAULT 0,           -- 0 = Todo (TaskItemStatus is an int underneath, Topic 1)
    CreatedAt DATETIME2(7)  NOT NULL
        CONSTRAINT DF_Tasks_CreatedAt DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_Tasks PRIMARY KEY (Id),
    -- Foreign key: every task must point to a project that EXISTS.
    CONSTRAINT FK_Tasks_Projects_ProjectId FOREIGN KEY (ProjectId) REFERENCES dbo.Projects (Id),
    -- Only the three TaskItemStatus values: 0 Todo, 1 InProgress, 2 Done.
    CONSTRAINT CK_Tasks_Status CHECK (Status IN (0, 1, 2))
);
GO

-- Index on the foreign key: makes "all tasks of project X" fast (Topic 18 goes deeper).
CREATE INDEX IX_Tasks_ProjectId ON dbo.Tasks (ProjectId);
GO

-- Check: both tables exist (empty for now).
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES ORDER BY TABLE_NAME;
```

- `USE ProjectHubDb;` switches the **current database**. Everything after it runs there. (Remember this line: Part 7 is mostly about it.)
- **Drop order:** `Tasks` first, because it references `Projects`. Dropping `Projects` first would fail with the FK error.
- **Create order:** `Projects` first, because the FK in `Tasks` needs it to exist.
- Every constraint has a **name** (`PK_`, `UQ_`, `DF_`, `FK_`, `CK_`) so errors are readable.
- No blank lines **inside** a statement: DBeaver can treat a blank line as "end of statement" (setup problem 5).

## 5.3 03-seed-data.sql

```
USE ProjectHubDb;
GO

INSERT INTO dbo.Projects (Name, Description, CreatedAt)
VALUES
    (N'Website redesign', N'New marketing site', '2026-09-01T09:00:00'),
    (N'Mobile app',       NULL,                  '2026-09-05T09:00:00'),
    (N'Internal tools',   N'Admin dashboards',   '2026-09-09T09:00:00');

-- Status: 0 Todo, 1 InProgress, 2 Done
INSERT INTO dbo.Tasks (ProjectId, Title, Status, CreatedAt)
VALUES
    (1, N'Design mockups',     2, '2026-09-02T09:00:00'),
    (1, N'Build landing page', 1, '2026-09-04T09:00:00'),
    (1, N'SEO audit',          0, '2026-09-06T09:00:00'),
    (1, N'Write copy',         2, '2026-09-03T09:00:00'),
    (2, N'Login screen',       1, '2026-09-05T09:00:00'),
    (2, N'Push notifications', 0, '2026-09-07T09:00:00'),
    (2, N'App store listing',  0, '2026-09-08T09:00:00');
GO

-- Check: 3 projects, 7 tasks.
SELECT * FROM dbo.Projects;
SELECT * FROM dbo.Tasks;
```

- The same data as `SampleData.cs`. One `INSERT` with several `VALUES` rows = one statement, one round trip.
- `Id` is not listed: IDENTITY generates it. Run right after 02 (fresh tables), so the ids are 1, 2, 3 and 1..7.
- Dates are written as ISO strings; SQL Server converts them to `DATETIME2`. `Status` is the enum's int.

## 5.4 04-queries.sql: the ten queries

```
USE ProjectHubDb;
GO

-- Q1. Everything from one table.                      API: GET /api/projects
SELECT * FROM dbo.Projects;

-- Q2. One row by its key, only some columns.          API: GET /api/projects/1
--     LINQ: projects.Where(p => p.Id == 1).Select(p => new { p.Id, p.Name, p.Description })
SELECT Id, Name, Description
FROM dbo.Projects
WHERE Id = 1;

-- Q3. Filter with several conditions: open tasks of project 1.
--     LINQ: tasks.Where(t => t.ProjectId == 1 && t.Status != TaskItemStatus.Done)
--     SQL: AND / OR / NOT, = and <> (not ==, not !=), IN (0, 1) for "one of these".
SELECT Id, Title, Status
FROM dbo.Tasks
WHERE ProjectId = 1 AND Status <> 2;

-- Q4. Sort: newest first, ties by title A-Z.
--     LINQ: tasks.OrderByDescending(t => t.CreatedAt).ThenBy(t => t.Title)
SELECT Id, Title, CreatedAt
FROM dbo.Tasks
ORDER BY CreatedAt DESC, Title ASC;

-- Q5. Paging: page 2 with 3 rows per page.
--     LINQ: tasks.OrderBy(t => t.Id).Skip(3).Take(3)       (OFFSET/FETCH requires ORDER BY)
SELECT Id, Title
FROM dbo.Tasks
ORDER BY Id
OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY;

-- Q6. Text search: names containing "app". % = any characters.
--     LINQ: projects.Where(p => p.Name.Contains("app"))   (case-insensitive here: default collation)
SELECT Id, Name
FROM dbo.Projects
WHERE Name LIKE N'%app%';

-- Q7. NULL: projects WITHOUT a description. Use IS NULL.
--     LINQ: projects.Where(p => p.Description == null)
SELECT Id, Name
FROM dbo.Projects
WHERE Description IS NULL;

-- Q7b. The classic trap: "= NULL" NEVER matches (0 rows), because NULL means "unknown",
--      and "unknown = unknown" is not true.
SELECT Id, Name
FROM dbo.Projects
WHERE Description = NULL;

-- Q8. INSERT a row and get the generated values back.  API: POST /api/projects
--     OUTPUT INSERTED.* returns the new row, like the 201 response body.
INSERT INTO dbo.Projects (Name, Description)
OUTPUT INSERTED.Id, INSERTED.Name, INSERTED.CreatedAt
VALUES (N'Docs portal', N'Public API docs');
```

(continued)

```
-- Q9. UPDATE: ALWAYS with a WHERE. Run the SELECT with the same WHERE first to see what you will change.
--     API: PUT /api/projects/{id}
SELECT Id, Name, Description FROM dbo.Projects WHERE Name = N'Docs portal';

UPDATE dbo.Projects
SET Name = N'Docs portal v2', Description = N'Renamed'
WHERE Name = N'Docs portal';

SELECT Id, Name, Description FROM dbo.Projects WHERE Name = N'Docs portal v2';

-- Q10. DELETE: also ALWAYS with a WHERE.               API: DELETE /api/projects/{id}
DELETE FROM dbo.Projects
WHERE Name = N'Docs portal v2';

-- Back to the 3 seed projects. No ORDER BY here, so the row order is NOT guaranteed
-- (in the cloud test they came back as 3, 2, 1). Need an order? Always write ORDER BY.
SELECT Id, Name FROM dbo.Projects;
```

- **Q8** `OUTPUT INSERTED.*` returns the new row, including the generated `Id` and the `DEFAULT CreatedAt`: the SQL version of a 201 response body.
- **Q9** shows the safety habit: SELECT, then UPDATE with the same WHERE, then SELECT again.
- **Re-runnable:** Q8 inserts, Q9 renames, Q10 deletes, so after a full run the table is back to the 3 seed projects (with a new gap in the ids).

## 5.5 05-constraints.sql: break it

```
USE ProjectHubDb;
GO

-- C1. Duplicate name, different case -> UNIQUE constraint (error 2627).   Our API: 409 Conflict
INSERT INTO dbo.Projects (Name) VALUES (N'MOBILE APP');
GO

-- C2. Task for a project that doesn't exist -> FOREIGN KEY (error 547).   Our API: 404 Not Found
INSERT INTO dbo.Tasks (ProjectId, Title) VALUES (999, N'Ghost task');
GO

-- C3. A status that isn't 0, 1 or 2 -> CHECK constraint (error 547).
INSERT INTO dbo.Tasks (ProjectId, Title, Status) VALUES (1, N'Weird status', 7);
GO

-- C4. Missing required value -> NOT NULL (error 515).                    Our API: 400 (automatic validation)
INSERT INTO dbo.Projects (Name, Description) VALUES (NULL, N'no name');
GO

-- C5. Delete a project that still has tasks -> FOREIGN KEY again (error 547): the tasks would point to nothing.
DELETE FROM dbo.Projects WHERE Id = 1;
GO

-- C6. Surprise: failed inserts still USED UP identity values. This valid insert does NOT get Id 4.
--     Lesson: never assume ids are consecutive. (Then it is deleted again, so the script stays re-runnable.)
INSERT INTO dbo.Projects (Name) OUTPUT INSERTED.Id, INSERTED.Name VALUES (N'Identity gap demo');
DELETE FROM dbo.Projects WHERE Name = N'Identity gap demo';
GO
```

## 5.6 scripts/run-sql.sh

```
#!/usr/bin/env bash
# Run one or more .sql files against the local SQL Server container (docker compose up -d first).
# Uses sqlcmd INSIDE the container, so nothing needs to be installed on the Mac.
# Usage (from the repo root):  ./scripts/run-sql.sh sql/01-create-database.sql sql/02-create-tables.sql sql/03-seed-data.sql
set -euo pipefail

PASSWORD="${MSSQL_SA_PASSWORD:-ProjectHub_Dev123!}"

for file in "$@"; do
  echo "=== $file"
  docker exec -i projecthub-sql /opt/mssql-tools18/bin/sqlcmd \
    -S localhost -U sa -P "$PASSWORD" -C -b -W < "$file"
done
```

- `docker exec -i projecthub-sql ...`: run a program **inside** the container. `-i` keeps stdin open, so `< "$file"` feeds the SQL file in.
- `sqlcmd` is SQL Server's own command-line client. It splits batches only on `GO`, exactly as SQL Server intends, which is why it's reliable for setup scripts.
- Flags: `-C` trust the self-signed certificate, `-b` stop with an error exit code when SQL fails, `-W` trim trailing spaces in output.
- `set -euo pipefail`: bash stops at the first failing command. `${MSSQL_SA_PASSWORD:-...}`: an env var, with the dev password as default.

# Part 6. Real results (SQL Server 2025, cloud container)

## 6.1 Setup scripts

Output of 03 (the two check SELECTs): 3 projects and 7 tasks. Mobile app's Description is NULL.

```
Changed database context to 'ProjectHubDb'.

(3 rows affected)

(7 rows affected)
Id Name Description CreatedAt
-- ---- ----------- ---------
1 Website redesign New marketing site 2026-09-01 09:00:00.0000000
2 Mobile app NULL 2026-09-05 09:00:00.0000000
3 Internal tools Admin dashboards 2026-09-09 09:00:00.0000000

(3 rows affected)
Id ProjectId Title Status CreatedAt
-- --------- ----- ------ ---------
1 1 Design mockups 2 2026-09-02 09:00:00.0000000
2 1 Build landing page 1 2026-09-04 09:00:00.0000000
3 1 SEO audit 0 2026-09-06 09:00:00.0000000
4 1 Write copy 2 2026-09-03 09:00:00.0000000
5 2 Login screen 1 2026-09-05 09:00:00.0000000
6 2 Push notifications 0 2026-09-07 09:00:00.0000000
7 2 App store listing 0 2026-09-08 09:00:00.0000000

(7 rows affected)
```

## 6.2 The ten queries

| # | Query | LINQ / API | Real result |
|---|---|---|---|
| Q1 | `SELECT * FROM dbo.Projects` | `GET /api/projects` | 3 rows |
| Q2 | `... WHERE Id = 1` | `GET /api/projects/1` | Website redesign |
| Q3 | `WHERE ProjectId = 1 AND Status <> 2` | `Where(t => ... && t.Status != Done)` | 2: Build landing page, SEO audit |
| Q4 | `ORDER BY CreatedAt DESC, Title ASC` | `OrderByDescending().ThenBy()` | 7 rows, App store listing first |
| Q5 | `OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY` | `Skip(3).Take(3)` | tasks 4, 5, 6 |
| Q6 | `WHERE Name LIKE N'%app%'` | `Contains("app")` | Mobile app |
| Q7 | `WHERE Description IS NULL` | `Description == null` | Mobile app |
| Q7b | `WHERE Description = NULL` | (the trap) | **0 rows** |
| Q8 | `INSERT ... OUTPUT INSERTED.Id ...` | `POST /api/projects` (201) | Id 4, CreatedAt from the DEFAULT |
| Q9 | SELECT, then `UPDATE ... WHERE Name = ...` | `PUT /api/projects/4` | renamed to "Docs portal v2" |
| Q10 | `DELETE ... WHERE Name = ...` | `DELETE /api/projects/4` | back to 3 projects, order **3, 2, 1** |

Full output of 04 (sqlcmd, -W). Note the empty result of Q7b and the 3, 2, 1 order at the end.

```
Changed database context to 'ProjectHubDb'.
Id Name Description CreatedAt
-- ---- ----------- ---------
1 Website redesign New marketing site 2026-09-01 09:00:00.0000000
2 Mobile app NULL 2026-09-05 09:00:00.0000000
3 Internal tools Admin dashboards 2026-09-09 09:00:00.0000000

(3 rows affected)
Id Name Description
-- ---- -----------
1 Website redesign New marketing site

(1 rows affected)
Id Title Status
-- ----- ------
2 Build landing page 1
3 SEO audit 0

(2 rows affected)
Id Title CreatedAt
-- ----- ---------
7 App store listing 2026-09-08 09:00:00.0000000
6 Push notifications 2026-09-07 09:00:00.0000000
3 SEO audit 2026-09-06 09:00:00.0000000
5 Login screen 2026-09-05 09:00:00.0000000
2 Build landing page 2026-09-04 09:00:00.0000000
4 Write copy 2026-09-03 09:00:00.0000000
1 Design mockups 2026-09-02 09:00:00.0000000

(7 rows affected)
Id Title
-- -----
4 Write copy
5 Login screen
6 Push notifications

(3 rows affected)
Id Name
-- ----
2 Mobile app

(1 rows affected)
Id Name
-- ----
2 Mobile app

(1 rows affected)
Id Name
-- ----

(0 rows affected)
Id Name CreatedAt
-- ---- ---------
4 Docs portal 2026-10-16 17:52:12.2603382
```

(continued)

```
(1 rows affected)
Id Name Description
-- ---- -----------
4 Docs portal Public API docs

(1 rows affected)

(1 rows affected)
Id Name Description
-- ---- -----------
4 Docs portal v2 Renamed

(1 rows affected)

(1 rows affected)
Id Name
-- ----
3 Internal tools
2 Mobile app
1 Website redesign

(3 rows affected)
```

## 6.3 The six refused statements

Full output of 05: five errors, then C6 succeeds with Id 7.

```
Changed database context to 'ProjectHubDb'.
Msg 2627, Level 14, State 1, Server 7a3b197acd25, Line 3
Violation of UNIQUE KEY constraint 'UQ_Projects_Name'. Cannot insert duplicate key in object 'dbo.Projects'. The duplicate key value is (MOBILE APP
The statement has been terminated.
Msg 547, Level 16, State 1, Server 7a3b197acd25, Line 3
The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Tasks_Projects_ProjectId". The conflict occurred in database "ProjectHubDb", ta
The statement has been terminated.
Msg 547, Level 16, State 1, Server 7a3b197acd25, Line 3
The INSERT statement conflicted with the CHECK constraint "CK_Tasks_Status". The conflict occurred in database "ProjectHubDb", table "dbo.Tasks", c
The statement has been terminated.
Msg 515, Level 16, State 2, Server 7a3b197acd25, Line 3
Cannot insert the value NULL into column 'Name', table 'ProjectHubDb.dbo.Projects'; column does not allow nulls. INSERT fails.
The statement has been terminated.
Msg 547, Level 16, State 1, Server 7a3b197acd25, Line 3
The DELETE statement conflicted with the REFERENCE constraint "FK_Tasks_Projects_ProjectId". The conflict occurred in database "ProjectHubDb", tabl
The statement has been terminated.
Id Name
-- ----
7 Identity gap demo

(1 rows affected)

(1 rows affected)
```

| # | Statement | Error | In our API |
|---|---|---|---|
| C1 | `INSERT ... (N'MOBILE APP')` | **2627** UNIQUE `UQ_Projects_Name` | **409** Conflict |
| C2 | task with `ProjectId = 999` | **547** FOREIGN KEY | **404** (the parent is in the URL: `/projects/999/tasks`) |
| C3 | task with `Status = 7` | **547** CHECK `CK_Tasks_Status` | **400** |
| C4 | project with `Name = NULL` | **515** NULL into a NOT NULL column | **400** (and our validation missed it) |
| C5 | `DELETE` project 1 (has tasks) | **547** REFERENCE constraint | **409** (or delete tasks first / cascade) |
| C6 | a valid insert after the failures | succeeds, **Id 7** | ids are not consecutive |

**Reading errors**

**How to read an SQL Server error:** (1) **the number**: 2627 / 2601 = duplicate, 547 = FK or CHECK, 515 = NULL, 208 = unknown table; (2) **the constraint name**: tells you which rule; (3) **the table and column**. "The statement has been terminated" = **nothing** was changed by that statement.

**Topic 10 preview:** with EF Core these come back wrapped in a `DbUpdateException` whose inner `SqlException.Number` is 2627 / 547 / 515. Mapping them to 409 / 404 / 400 is exactly what we did by hand in Topic 6 with our own exceptions.

# Part 7. Setup, and the 8 problems we hit

## 7.1 Docker: SQL Server in a container

**Docker** runs a program (SQL Server, a Linux program) in an isolated **container** on your Mac, so you don't install it. `docker-compose.yml` in the repo describes which container to run. TS analogy: `package.json` + `npm start`, but for a whole server.

1. Open **Docker Desktop**, wait for **Engine running**. On an Apple chip, Settings, General: **Use Rosetta for x86_64/amd64 emulation** on (SQL Server only exists for Intel; our compose file says `platform: linux/amd64`).
2. In the repo root: `docker compose up -d` (`-d` = in the background). The first time it downloads the image.
3. `docker ps`: a line with `projecthub-sql`, status `Up ...`.
4. `docker logs projecthub-sql`: wait for **SQL Server is now ready for client connections**.

| Command | What it does |
|---|---|
| `docker compose up -d` | start SQL Server |
| `docker compose stop` | stop it (data kept) |
| `docker compose down` | remove the container (data **kept** in the `sqldata` volume) |
| `docker compose down -v` | remove everything **including the data** (fresh start; also resets the sa password) |

## 7.2 DBeaver: the connection

**DBeaver** is a database client (like an IDE for databases). New Database Connection, SQL Server: Host `localhost`, Port `1433`, Authentication **SQL Server Authentication**, user `sa`, password `ProjectHub_Dev123!` (dev only), tick **Save password**. The first Test Connection downloads the driver. Certificate error: Driver properties, `trustServerCertificate = true`.

## 7.3 The 8 problems

Each one with what you saw, the cause, the fix, and the lesson. Read them as a debugging story: the symptoms looked different, but problems 3 to 8 share one root.

### 1. "Login failed for user 'sa'"

| | |
|---|---|
| You saw | Test Connection after the driver download. |
| Cause | A **password typo**. The driver and network were fine: a server answered. |
| Fix | Edit Connection (F4), retype the password by hand. |
| Lesson | Servers deliberately say only "Login failed" (no reason) so attackers can't tell what was wrong. The real reason is in the **server log**: `docker logs projecthub-sql \| grep -i "login failed"` ("Password did not match"). If no line appears, you reached **another** server on port 1433. |

### 2. "Cannot obtain session: No active connection"

| | |
|---|---|
| You saw | Running 01. The tab title said `<none> 01-create-database.sql`. |
| Cause | Files opened with File, Open File are **not linked** to a connection. |
| Fix | Click in the editor, SQL Editor, **Set active connection**, pick the connection. The tab becomes `<master> ...`. |
| Lesson | A SQL file is just text; the **connection** decides which server it goes to. |

### 3. "Script 1 did nothing, so script 2 failed" (Error 911: database does not exist)

| | |
|---|---|
| You saw | 01 seemed to run, 02 failed. |
| Cause | The top ▶ is **Execute SQL Statement** (only the statement under the cursor). 01's comment line was "the statement". |
| Fix | The second ▶ with a small + is **Execute SQL Script** (the whole file). You found it yourself. |
| Lesson | Statement vs script: know what your Run button sends. |

### 4. Q1 returned columns but 0 rows

| | |
|---|---|
| You saw | `SELECT * FROM dbo.Projects` showed headers, no data. |
| Cause | At the time I guessed "03 didn't run, or 02 re-ran after 03". **The real cause was found only in problem 7.** |
| Fix | (Re-running 02 and 03 didn't help, which was the clue.) |
| Lesson | An empty result is also a result. "Which table did this actually read?" is the first question. |

### 5. 03 failed: Error 208 "Invalid object name 'dbo.Tasks'"

| | |
|---|---|
| You saw | Projects seemed to exist, Tasks didn't. |
| Cause | My guess: blank lines inside `CREATE TABLE` made DBeaver split the statement. **Real cause (problem 7):** 03 ran in master, where only a stray `Projects` existed. |
| Fix | Removed blank lines inside statements (harmless, kept). |
| Lesson | **My mistake:** I tested with sqlcmd and assumed DBeaver behaves the same. Test with the tool the user uses. |

### 6. Error 102 "Incorrect syntax near 'Projects'"

| | |
|---|---|
| You saw | DBeaver's Execute SQL Script sent only `CREATE TABLE dbo.Projects` (the first line). |
| Cause | DBeaver's script parser split the multi-line statement. Exact cause unknown (no DBeaver in the cloud). |
| Fix | **Stop guessing:** setup scripts run with sqlcmd through `scripts/run-sql.sh`; DBeaver is for browsing and for 04/05 one statement at a time. |
| Lesson | When a tool is unreliable for a job, switch tools instead of fighting it. |

### 7. "omg": tables exist in ProjectHubDb, Q1 still empty, toolbar says `dbo@master`

| | |
|---|---|
| You saw | The navigator showed ProjectHubDb with both tables; the editor's toolbar showed `dbo@master`. |
| Cause | **The root cause of 4–7.** The **first** DBeaver run of 02 failed at `USE ProjectHubDb` (it didn't exist yet), but DBeaver **continued in master** and created an empty `master.dbo.Projects`. From then on Q1 read that empty table, and 03 failed because master had Projects but no Tasks. |
| Fix | Edit Connection, Database: `ProjectHubDb`; drop the stray tables in master; run 01–03 with run-sql.sh. |
| Lesson | Always know **which database** a query runs in. |

### 8. "No result still, but there is data"

| | |
|---|---|
| You saw | Navigator, Projects, Data tab: 3 rows. Editor Q1: 0 rows. Toolbar still `dbo@master`. |
| Cause | The Data tab reads the table you clicked (`ProjectHubDb.dbo.Projects`). The editor ran `dbo.Projects` in **its** current database: master. Ctrl+Enter ran only Q1, not the `USE` line above it; editing the connection doesn't change editor tabs that are already open; and the stray master table still existed. |
| Fix | Toolbar `dbo@master`, pick ProjectHubDb / dbo (or run the `USE` line first); check with `SELECT DB_NAME();`; drop the stray tables. |
| Lesson | Two-part names (`dbo.Projects`) are resolved in the **current** database; three-part names (`ProjectHubDb.dbo.Projects`) work from anywhere. |

## 7.4 The current database, reproduced

I reproduced problems 7 and 8 in the cloud: put a stray empty `Projects` table in master, query it, then drop it:

docs/studybooks/notes/08-master-trap.txt

```
-- Reproduction of setup problems 7-8 (cloud, sqlcmd): a stray empty dbo.Projects in master
USE master;
GO
SELECT DB_NAME() AS CurrentDb;
-- what the failed first DBeaver run of 02 left behind:
CREATE TABLE dbo.Projects (Id INT IDENTITY(1,1) PRIMARY KEY, Name NVARCHAR(100) NOT NULL, Description NVARCHAR(1000) NULL, CreatedAt DATETIME2(7) N
GO
SELECT * FROM dbo.Projects;                  -- the empty stray table (your "omg" / "no result still")
SELECT COUNT(*) AS RealRows FROM ProjectHubDb.dbo.Projects;   -- three-part name works from any database
DROP TABLE dbo.Projects;
GO
SELECT * FROM dbo.Projects;                  -- now it fails loudly instead of lying
GO
-- ---- output ----
Changed database context to 'master'.
CurrentDb
---------
master

(1 rows affected)
Id Name Description CreatedAt
-- ---- ----------- ---------

(0 rows affected)
RealRows
--------
3

(1 rows affected)
Msg 208, Level 16, State 1, Server 7a3b197acd25, Line 1
Invalid object name 'dbo.Projects'.
```

**The lesson of Part 7**

**Every connection has a current database.** If you don't set one, it's the login's default: usually **master**, the server's system database.

`SELECT DB_NAME();` = "where am I?". Run it whenever results look wrong.

A stray empty table **lies quietly** (0 rows). A missing table **fails loudly** (Msg 208). Loud failures are easier to debug.

In C#, the current database comes from the **connection string**: `Server=localhost,1433;Database=ProjectHubDb;...`. EF Core (Topic 10) always connects with it, so the app can't land in master by accident.

Same lesson as Topic 5's stale build: **check the context first** (which server, which database, which build) before debugging the logic.

# Part 8. Investigating without a debugger

SQL has no breakpoints. You investigate with the tools: error messages, small check queries, the server log, and the navigator. You ran 04 and 05 and chose to skip the DEBUG investigation, so here are the five tasks **with answers** (verified in the cloud).

| Task | Answer |
|---|---|
| 1. Read the errors of 05: number, constraint, matching C# rule | C1 2627 `UQ_Projects_Name` = Topic 3 duplicate name (ConflictException, 409). C2 547 FK = NotFoundException for the project (404). C3 547 `CK_Tasks_Status` = only valid enum values (400). C4 515 = required Name (400). C5 547 REFERENCE = can't delete a project with tasks (409). |
| 2. NULL experiment: does `WHERE Description <> N'x'` return Mobile app? | **No.** Only projects 1 and 3. `NULL <> 'x'` is UNKNOWN, and WHERE keeps only TRUE. To include it: `... OR Description IS NULL`. |
| 3. Order experiment: `SELECT Id, Name FROM dbo.Projects` vs `... ORDER BY Id` | Without ORDER BY we got 3, 2, 1 (the name index was cheapest to read). Only the ORDER BY version is trustworthy in a UI. |
| 4. ER Diagram of Tasks: which side is "many"? | **Tasks**: the FK line goes from `Tasks.ProjectId` to `Projects.Id`. The FK always lives on the many side. |
| 5. Your own query: Todo tasks of project 2, oldest first | See below. Result: 6 Push notifications (09-07), 7 App store listing (09-08). |

Task 5 / re-check (c): you asked me to write it

```
SELECT Id, Title, CreatedAt
FROM dbo.Tasks
WHERE ProjectId = 2 AND Status = 0      -- 0 = Todo (the C# enum value is stored as an int)
ORDER BY CreatedAt ASC;                 -- ASC is the default; oldest first

-- LINQ: tasks.Where(t => t.ProjectId == 2 && t.Status == TaskItemStatus.Todo).OrderBy(t => t.CreatedAt)
-- Topic 9: use the project NAME instead of 2 with a JOIN.
```

## Investigation toolbox

| Question | Tool |
|---|---|
| Which database am I in? | `SELECT DB_NAME();`, the DBeaver toolbar (`dbo@...`) |
| Is the server up? Why did login fail? | `docker ps`, `docker logs projecthub-sql` |
| What exactly did DBeaver send? | the query text at the top of the result tab |
| Which rows will my UPDATE/DELETE touch? | the same WHERE as a SELECT first; "(N rows affected)" |
| Does the table really have data? | navigator, table, Data tab (reads that exact table) |
| What rules does a table have? | navigator, table, Constraints / Foreign Keys / ER Diagram tabs |

# Part 9. Your answers, corrections, interview Q&A

## 9.1 CHECK: your answers

| Question | Your answer | Verdict / complete answer |
|---|---|---|
| 1. Table, row, column, PK, FK | table: projects/tasks; row: one unit (a project/task); column: property; PK: id; FK: projectId, link to another table's item | **Right.** Sharpen: PK = unique + never NULL; FK = value must exist as a PK in the other table, enforced (C2) |
| 2. Why no tasks column? | tables don't work like this; relation is in the ProjectId column of Tasks | **Right.** + a column holds one value; the FK lives on the many side |
| 3. IDENTITY(1,1) | nextId++ (unique, next one) | **Half.** + generated **atomically by the database** (safe across servers, unlike our lock); id known only after INSERT; gaps are normal (C6: 7) |
| 4. UNIQUE + service check: two reasons | last DB check; no one can insert it even directly in the DB | **One of two.** Re-check: "servers concurrency" (**right**) |
| 5. NVARCHAR vs VARCHAR | "answer, be very brief" | Unicode (any language, 2 bytes/char) vs a 1-byte code page. Use NVARCHAR + N'...' |
| 6. `= NULL` | nothing is = to null; IS NULL | **Right.** + it's UNKNOWN (not false); WHERE keeps only TRUE; `<>` has the same trap |
| 7. Why ORDER BY | otherwise the order will be random | **Almost:** undefined, not random; looks stable, then changes (our 3, 2, 1) |
| 8. UPDATE/DELETE danger + habit | without WHERE they delete everything | **Half:** the habit is SELECT with the same WHERE first, check the count, then change |
| 9. 2627 / 547 / 515 to HTTP | "doesn't matter for me now"; re-check: 2627 = 409 (**right**) | 2627 409; 547 FK 404 (parent in URL) or 400 (in body); 515 400. Matters in Topic 10 (DbUpdateException) |
| 10. SQL injection | user input directly in the SQL query; params prevent it | **Right.** + text and values travel separately; escaping by hand isn't a fix; EF always parameterizes |
| Re-check (c): Todo tasks of project 2 | "u pls" | written in Part 8 |

## 9.2 Mistakes and corrections in this topic

- **You:** "an FK is an index". Correction: FK = a rule (correctness), index = speed; SQL Server doesn't index FKs automatically (Part 2.3).
- **You:** "ORDER BY, otherwise random". Correction: **undefined**, which is worse, because it looks stable until it isn't.
- **Me:** problems 4 and 5 were misdiagnosed ("03 didn't run", "blank lines"). The real cause was the current database (master). Lesson for both of us: check the context first.
- **Me:** I tested the scripts only with sqlcmd and assumed DBeaver would behave the same. It didn't (problems 5, 6).

## 9.3 Interview questions with model answers

**Q: Table, row, column, primary key, foreign key?**

A table holds rows of the same shape; a row is one record (one project); a column is a typed field (Name). The PK uniquely identifies a row and is never NULL (Projects.Id). An FK is a column holding another table's PK value plus a rule that the row must exist (Tasks.ProjectId to Projects.Id).

**Q: How is a one-to-many relationship stored?**

With an FK column on the many side: Tasks.ProjectId. The one side (Projects) stores nothing; "tasks of project 1" is WHERE ProjectId = 1.

**Q: What does IDENTITY do?**

The database generates the key (start, step) atomically on INSERT, so concurrent inserts never collide. You read it back with OUTPUT INSERTED.Id (EF does it for you). Gaps are normal.

**Q: Is a foreign key an index?**

No. An FK is a constraint (correctness); an index is a lookup structure (speed). SQL Server auto-indexes PK and UNIQUE, not FKs, so you usually add an index on FK columns.

**Q: Why enforce uniqueness in the database if the code already checks?**

Other write paths (imports, scripts, other apps) and concurrency: two servers can both pass the code check; only the database sees every insert. It's also the last line of defense.

**Q: NVARCHAR vs VARCHAR?**

NVARCHAR is Unicode (any language); VARCHAR uses a single-byte code page. Use NVARCHAR and N'...' literals for user text.

**Q: Why does WHERE x = NULL return nothing?**

Comparisons with NULL are UNKNOWN under three-valued logic, and WHERE keeps only TRUE rows. Use IS NULL / IS NOT NULL.

**Q: Why must a UI query have ORDER BY?**

A table is an unordered set; without ORDER BY the order is undefined and can change with data, indexes or versions. Paging also requires ORDER BY.

**Q: In what order is a SELECT evaluated?**

Logically FROM, WHERE, (GROUP BY, HAVING,) SELECT, ORDER BY, then OFFSET/FETCH. That's why a SELECT alias can't be used in WHERE but can be in ORDER BY.

**Q: What's dangerous about UPDATE/DELETE? Your habit?**

Without WHERE they hit every row and there's no undo. Run a SELECT with the same WHERE first, check the row count, then change; use a transaction for bigger changes.

**Q: Map 2627, 547 and 515 to HTTP status codes.**

2627 duplicate key: 409. 547 FK on insert: 404 if the parent id is in the URL, 400 if it's in the body; 547 on delete (still referenced): 409; 547 CHECK: 400. 515 NOT NULL: 400.

**Q: What is SQL injection? How do you prevent it?**

User input concatenated into SQL text becomes code. Prevent it with parameters: text and values are sent separately. EF Core and Dapper parameterize by default.

**Q: What is GO?**

Not SQL: a batch separator understood by tools (sqlcmd, SSMS, DBeaver). The text above it is sent to the server as one batch.

**Q: Your query returns no rows but the table has data. What do you check?**

The context: SELECT DB_NAME() (am I in the right database?), the server/connection, then the WHERE (NULL comparisons, case, types). In our setup it was the master database.

**Q: Why aren't identity values consecutive?**

Failed inserts and rollbacks consume values, deleted ids are never reused, and the server caches values. Never rely on consecutive ids.

# Part 10. Summary

**Key takeaways**

**Tables, rows, columns** with fixed types; the database enforces the shape and the rules for **everyone**.
**PK** identifies a row (IDENTITY: generated, atomic, gaps normal). **FK** = the relationship, on the many side, plus an existence rule. FK is not an index.
**Constraints** (NOT NULL, UNIQUE, CHECK, DEFAULT, FK) duplicate C# rules on purpose: other write paths + concurrency across servers.
**Queries:** FROM, WHERE, SELECT, ORDER BY; `IS NULL`, never `= NULL`; always `ORDER BY` for lists; `OFFSET/FETCH` = Skip/Take.
**Changes:** SELECT with the same WHERE before UPDATE/DELETE; `OUTPUT INSERTED` returns generated values.
**Errors:** number + constraint name + table. 2627 = duplicate, 547 = FK/CHECK, 515 = NULL, 208 = unknown object.
**Security:** parameters, never string concatenation.
**Context first:** which database am I in? `SELECT DB_NAME()`. Our whole setup saga was a query running in master.

## Cheat sheet

| Need | SQL |
|---|---|
| read everything | `SELECT * FROM dbo.Projects;` |
| filter + some columns | `SELECT Id, Title FROM dbo.Tasks WHERE ProjectId = 1 AND Status <> 2;` |
| sort | `ORDER BY CreatedAt DESC, Title ASC` |
| page | `ORDER BY Id OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY` |
| search text | `WHERE Name LIKE N'%app%'` |
| missing value | `WHERE Description IS NULL` |
| insert + get id | `INSERT INTO dbo.Projects (Name) OUTPUT INSERTED.Id VALUES (N'x');` |
| update | `UPDATE dbo.Projects SET Name = N'y' WHERE Id = 4;` (SELECT first!) |
| delete | `DELETE FROM dbo.Projects WHERE Id = 4;` (SELECT first!) |
| where am I? | `SELECT DB_NAME();` / `USE ProjectHubDb;` |
| run a script file | `./scripts/run-sql.sh sql/01-create-database.sql ...` |
| start / stop SQL Server | `docker compose up -d` / `docker compose stop` |

## Checklist: can you do this without notes?

- Draw Projects and Tasks with their PK, FK and constraints, and explain why the FK is on Tasks.
- Explain IDENTITY and why our C# `lock` + `_nextId++` doesn't survive multiple servers.
- Give both reasons for a UNIQUE constraint next to the C# check.
- Write: Todo tasks of project 2, oldest first. Then page 2 of all tasks, 3 per page.
- Explain why `= NULL` and `<> 'x'` both skip NULL rows.
- Name the errors 2627, 547, 515 and their HTTP status codes.
- Explain SQL injection and parameters in two sentences.
- Find out which database a query runs in, and fix it in DBeaver.

## What comes next: Topic 9 (JOINs, GROUP BY, transactions)

Queries across **both** tables: `JOIN` (tasks with their project names), `GROUP BY` / `HAVING` ("tasks per project", "open tasks per status"), subqueries and `EXISTS`, and **transactions** (`BEGIN TRAN` ... `ROLLBACK`), the real safety net for UPDATE/DELETE. Checkpoint: explain why a JOIN is needed and what a transaction protects.